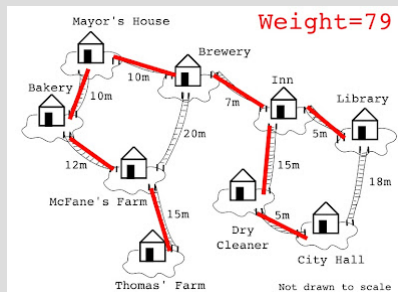


Minimalne drzewo rozpinające

Piotr Pawlik

Do czego można zastosować ?

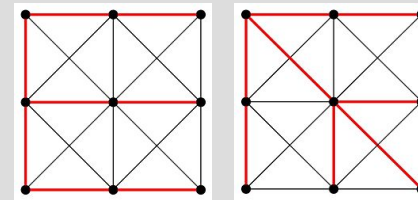
- Infrastruktura - połączenie czegoś (domy, miasta, skrzyżowania) czymś (droga, przewód, rurociąg).
- Ogólnie - projektowanie sieci.
- Segmentacja



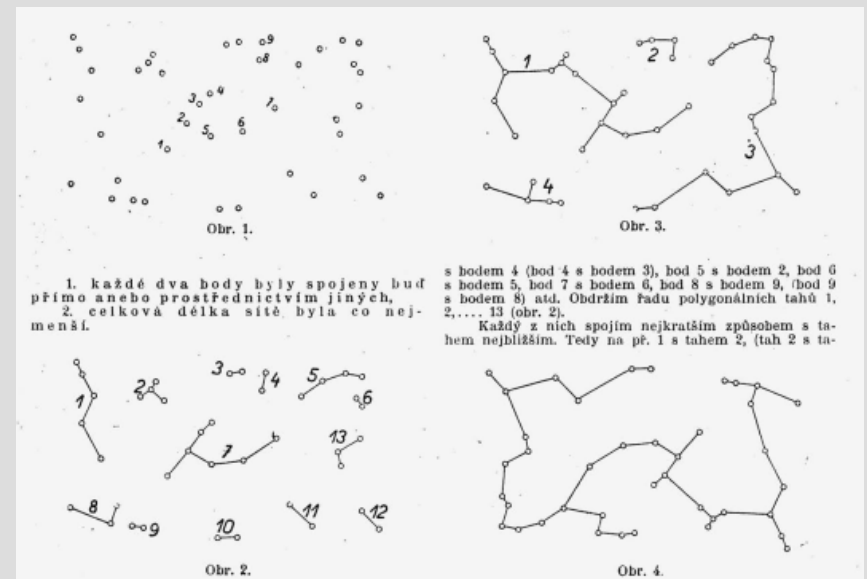
Drzewa rozpinające

Drzewo rozpinające - drzewo, które zawiera wszystkie wierzchołki nieskierowanego grafu G , zaś zbiór krawędzi drzewa jest podzbiorem zbioru krawędzi grafu.

Minimalne drzewo rozpinające MST/SST (*minimal spanning tree, shortest spanning tree*) - drzewo rozpinające danego grafu o najmniejszej z możliwych wag, tj. takie, że nie istnieje dla tego grafu inne drzewo rozpinające o mniejszej sumie wag krawędzi.



Algorytm Borůvky



Algorytm Borůvky

W każdym wierzchołku incydentna krawędź z najmniejszą wagą ('najlżejsza') należy też do MST

MST = []

Dla każdego wierzchołka v w grafu G

Znajdź 'najlżejszą' krawędź i dołącz ją do MST

Znalezione krawędzie łączą wierzchołki w podgrafy - zastąp każdy z nich jednym nowym wierzchołkiem w nowym grafie G'

Dla grafu G' powtórz to, co zrobiono dla grafu G

Powtarzaj aż G' nie będzie jednym wierzchołkiem

Algorytm Borůvky Implementacja z G'

Graf G - lista sąsiedztwa

Stworzenie pustej listy krawędzi jako MST

Stworzenie pustego grafu G'

Dla każdego wierzchołka G :

Wyszukiwanie w liście 'najlżejszej' krawędzi

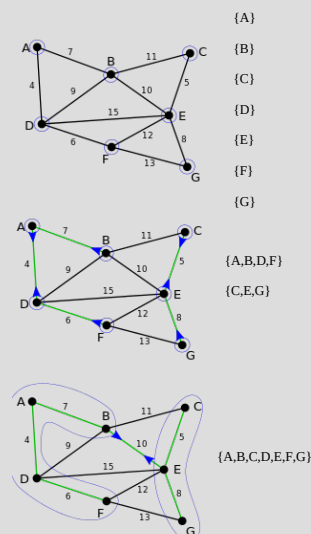
Dołączenie jej do MST

Dodanie do G' wierzchołka z połączoną listą sąsiadów
lub dołączenie sąsiada do istniejącego wierzchołka
(trzeba usunąć krawędzie „wewnętrzne” i duplikaty)

$G = G'$

Powtórzenie dopóki G' zawiera więcej niż jeden wierzchołek

Algorytm Borůvky



http://en.wikipedia.org/wiki/Borůvka's_algorithm

Złożoność:

W każdym obiegu pętli sprawdzane jest max. E krawędzi

Po każdym obiegu liczba wierzchołków maleje w najgorszym razie dwukrotnie, a więc obiegów będzie max.

$\log V$

Stąd złożoność:

$O(E \log V)$

Algorytm Borůvky Implementacja bez G'

Graf G - lista sąsiedztwa

Stworzenie pustej listy krawędzi jako MST

~~Stworzenie pustego grafu G'~~

Dla każdego wierzchołka G :

Wyszukiwanie w liście 'najlżejszej' krawędzi (**inny** **komponent**)

Dołączenie jej do MST

Połączenie **komponentów**

Powtórzenie dopóki istnieje więcej niż jeden **komponent**

Algorytm Borůvky

Implementacja bez G'

Jak to zrobić bez G'?

komponent - 'drzewo rodziców' w postaci listy

find - sprawdza do jakiego komponentu należy wierzchołek

union - łączy dwa komponenty

„Wyszukiwanie w liście 'najlżejszej' krawędzi (inny komponent)”:
- wołanie **find** na wierzchołkach incydentnych, szukanie
'najlżejszej' wśród należących do innych komponentów niż
aktualny wierzchołek

Problem Find-Union

Możliwe implementacje:

- można poetykietować wierzchołki - wtedy łatwo sprawdzić, czy dwa są w tym samym pod-drzewie (find zwraca etykietę w czasie $O_{\text{find}}(1)$). Niestety, łączenie wymaga przetykietowania wierzchołków (czas $O_{\text{union}}(V)$).
- można traktować tworzone MST jako zbiór komponentów (podgrafów). Wtedy łącznie 'robi się samo' (wstawienie krawędzi do MST – czas $O_{\text{union}}(1)$)
Wtedy jednak jest problem z identyfikacją komponentu – czas $O_{\text{find}}(V)$.

Algorytm Borůvky

Implementacja bez G'

Graf G - lista sąsiedztwa

Stworzenie pustej listy krawędzi jako MST

~~Stworzenie pustego grafu G'~~

Dla każdego **wierzchołka** z G:

Wyszukiwanie w liście 'najlżejszej' krawędzi (**find**)

Dołączenie jej do MST

Połączenie komponentów (**union**)

Powtórzenie dopóki istnieje więcej niż jeden komponent

Tym razem złożoność:

$O(\log V * (E * O_{\text{find}}(??) + O_{\text{union}}(??)))$

Problem Find-Union

Propozycja lepszego rozwiązania.

Reprezentacja komponentów jako zbiorów – tzw.

struktura zbiorów rozłącznych (ustalony zbiór i podział na mniejsze, rozłączne zbiory (poddrzewa)).

Operacje:

- find - czy element jest w zbiorze
- union - łączymy dwa zbiory

Problem Find-Union

- Budujemy drzewo “wstecze” - poszczególne wierzchołki wskazują w stronę korzenia.
- Etykieta korzenia jest jednocześnie etykietą całego drzewa.
- Dodatkowo każdy wierzchołek “pamięta” rozmiar poddrzewa.

Nasze dwie operacje:

- find(v) - znajdź korzeń (root) dla v => rekurencyjne przejście po rodzicach w $O(\log V)$, (sprawdzenie czy dwa wierzchołki są w tym samym poddrzewie – $2 * \text{find}$)
- union(u,v) - łączenie u i v => podpięcie korzenia dla u pod korzeń dla v (lub na odwrót - zależy które poddrzewo jest wyższe) w $O(1)$

Algorytm Borůvky

Utwórz komponent z każdego wierzchołka

Dla każdego **komponentu**:

Znajdź ‘najlżejszą’ krawędź należącą do innego komponentu (**find**)

Połącz te komponenty (**union**)

Algorytm Borůvky Implementacja bez G'

Graf G - lista sąsiedztwa

Stworzenie pustej listy krawędzi jako MST

~~Stworzenie pustego grafu G'~~

Dla każdego **wierzchołka** z G:

Wyszukiwanie w liście ‘najlżejszej’ krawędzi (**find**)

Dołączenie jej do MST

Połączenie komponentów (**union**)

Powtórzenie dopóki istnieje więcej niż jeden komponent

Tym razem złożoność:

$$O(\log V * (E * O_{\text{find}}(\log V) + O_{\text{union}}(1))) = O(\log V E \log V) = O(E (\log V)^2)$$

Algorytm Kruskala

Utwórz komponent z każdego wierzchołka

Posortuj wszystkie krawędzie rosnąco

Dla każdej **krawędzi**:

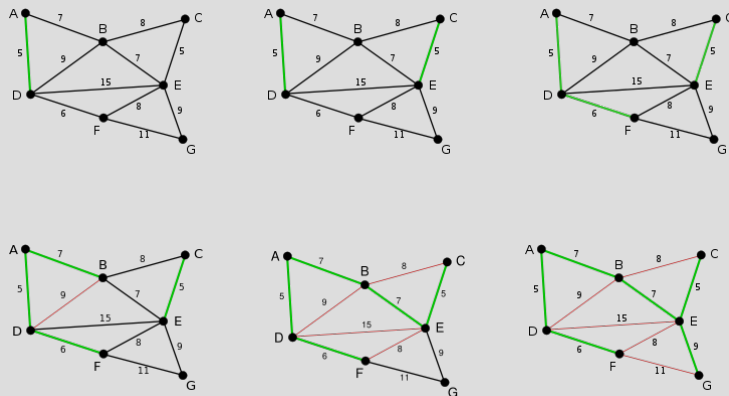
Jeżeli wierzchołki do niej incydentne należą do różnych komponentów (**find**)

To połącz te komponenty (**union**)

Tym razem złożoność:

$$O(V) + O(E \log E) + O(E * (\log V + O(1))) = O(E \log E) + O(E \log V) = O(E \log E)$$

Algorytm Kruskala



AD	5
CE	5
DF	6
AB	7
BE	7
BC	8
EF	8
DB	9
EG	9
FG	11
DE	15

Źródło: https://pl.wikipedia.org/wiki/Algorytm_Kruskala

‘Jednokomponentowy’ algorytm Borůvki

Dodaj dowolny v do MST

Wśród **wierzchołków** z MST:

Wyszukaj ‘najlżejszą’ krawędź łączącą wierzchołek z MST z wierzchołkiem spoza.
Dołącz ją do MST (z wierzchołkiem)

Powtórzenie dopóki są wierzchołki nie należące do MST

Tym razem złożoność:

$$O(V * (V/2 * E/V)) = O(V * E)$$

Algorytm Borůvki Implementacja bez G'

Graf G - lista sąsiedztwa

Stworzenie pustej listy krawędzi jako MST

~~Stworzenie pustego grafu G'~~

Dla każdego **wierzchołka** z G:

Wyszukiwanie w liście ‘najlżejszej’ krawędzi (inny komponent)

Dołączenie jej do MST

Połączenie komponentów

Powtórzenie dopóki istnieje więcej niż jeden komponent

Tym razem złożoność:

$$O(\log V * (E * O_{\text{find}}(\log V) + O_{\text{union}}(1))) =$$

$$O(E (\log V)^2)$$

Algorytm Prima

Dodaj dowolny v do MST

~~Wśród wierzchołków z MST:~~

→ Dla każdego jego sąsiada vs spoza MST:

$$\text{min_dist} = \min(\text{dist}(v, v_s), \text{min_dist})$$

Znajdź nowy v spoza MST z najmniejszą min_dist i dołącz v do MST (z krawędzią)

Powtórzenie dopóki są wierzchołki nie należące do MST

Tym razem złożoność:

$$O(V * ((E/V * O_{\text{update}}) + O_{\text{find_min}}))$$

Algorytm Prima

Dodaj dowolny v do MST

Wśród ~~wierzchołków~~ z MST:—

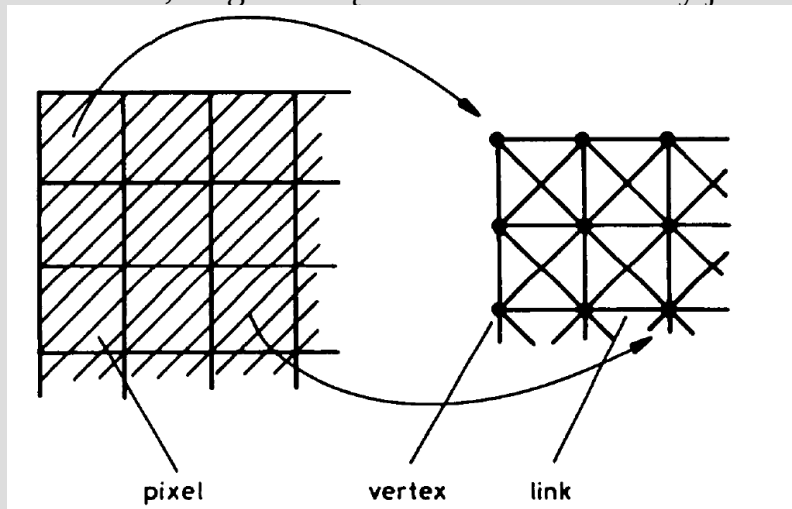
- Dla każdego jego sąsiada vs spoza MST:
 $\text{min_dist} = \min(\text{dist}(v, vs), \text{min_dist})$
Znajdź nowy v spoza MST z najmniejszą min_dist i dołącz v do MST (z krawędzią)
- Powtórzenie dopóki są wierzchołki nie należące do MST

Tym razem złożoność – lista min_dist :

$$O(V * ((E/V * (1)) + (V))) = O(E + V^2)$$

Zastosowania w przetwarzaniu obrazów

Graf rzadki, waga krawędzi to moduł z różnicy jasności



[Graph theory for image analysis: an approach based on the shortest spanning tree]

Algorytm Prima

Dodaj dowolny v do MST

Wśród ~~wierzchołków~~ z MST:—

- Dla każdego jego sąsiada vs spoza MST:
 $\text{min_dist} = \min(\text{dist}(v, vs), \text{min_dist})$
Znajdź nowy v spoza MST z najmniejszą min_dist i dołącz v do MST (z krawędzią)
- Powtórzenie dopóki są wierzchołki nie należące do MST

Tym razem złożoność – **kopiec** min_dist :

$$O(V * ((E/V * (\log V)) + (\log V))) = O(E \log N)$$

Zastosowania w przetwarzaniu obrazów

Segmentacja - podział obrazu na obszary jednorodne pod względem jakiegoś kryterium. Jeżeli obraz zapisać jako graf $G = (V, E)$

W tym przypadku podział obrazu na komponenty A_i , z których każdy jest grafem $G' = (V', E')$ takim, że $V' \subseteq V$, $E' \subseteq E$ oraz E' zawiera tylko wierzchołki z V' .

Inaczej ujmując problem: $A_1, \dots, A_k$ to taki podział grafu G , że:

$$A_i \cap A_j = \emptyset \quad (i, j \in \{1, 2, \dots, k\}, i \neq j)$$

$$A_1 \cup \dots \cup A_k = G$$

Zastosowania w przetwarzaniu obrazów

Idea: odpowiedni podział drzewa - powstaje “las”, który także rozpina graf (każdy piksel należy do jakiegoś drzewa).

Podziału dokonujemy na podstawie krawędzi o największych wagach

Właściwości:

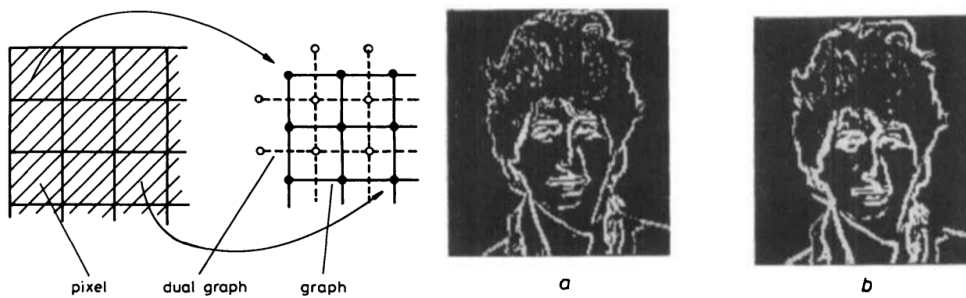
- uwzględniania zależności przestrzenne a nie ilościowe,
- dokładne odwzorowanie krawędzi obszarów,
- obszary są zamknięte, nie ma potrzeby stosowania śledzenia punktów krawędzi,
- możliwość podziału na dowolną liczbę obszarów na podstawie MST,
- kolejny podział wpływa tylko na dane drzewo - nie ma zmian już wyznaczonych krawędzi

Zastosowania w przetwarzaniu obrazów

Problemy:

- mała odporność na szумы (np. impulsowe),
- potencjalne łączenie obszarów o różnych jasnościach => problemem funkcja podobieństwa.

Wykorzystanie do znajdowania krawędzi:



Zastosowania w przetwarzaniu obrazów

Idea: odpowiedni podział drzewa - powstaje “las”, który także rozpina graf (każdy piksel należy do jakiegoś drzewa).

Podziału dokonujemy na podstawie krawędzi o największych wagach

- z obrazka budujemy graf,
- krawędzie zawierają moduł z różnicy jasności,
- znajdujemy MST i z niego usuwamy krawędź (w ogólności krawędzie) o największej wadze,
- wizualizacja:
trawersacja obu poddrzew i kolorowanie węzłów (pikseli) każdego innym kolorem



Algorytmy i struktury danych

To już jest koniec ...

Dziękuję za uwagę